

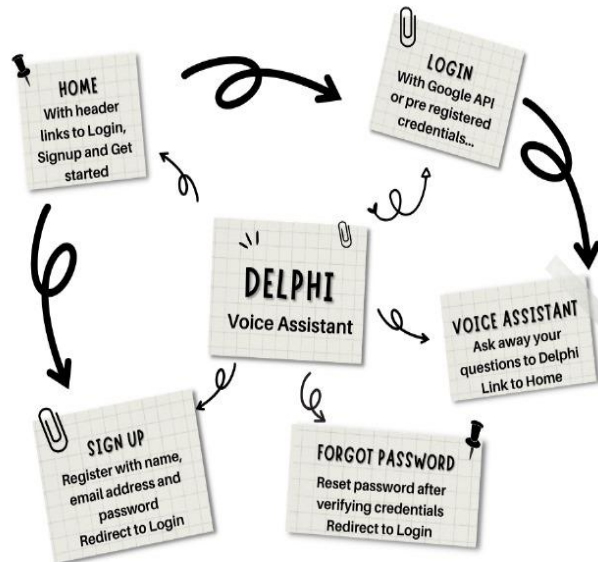
About the Project

In this project we have developed a Voice Assistant named Delphi. Delphi is your go to voice assistant and can perform all the basic tasks that any voice assistant like Siri, Alexa do, e.g. play this song for, tell me more about Paris or let us say tell me about weather. Our assistant provides you with the personalized ancient Greece aesthetics experience which will thrill you.

Functionalities of the project

Our project includes the following features:

- Login
- Signup
- Google Login
- Search on google
- Play music on YouTube
- Knowing the Time
- Knowing weather
- Extracting any other information



Technologies Used

Frontend: React JS, HTML, CSS

React JS:

It is a free, open-source JavaScript library used for building user interfaces using components. React uses JSX, a syntax extension to JavaScript, which provides a way to structure component rendering using syntax familiar to many developers. React uses Virtual DOM(Document Object Model), that allows changes to be implemented easily. ReactJS is a declarative, effective and flexible JavaScript library.

A ReactJS application is made up of multiple components, each component responsible for outputting a small, reusable piece of HTML code. These components can be nested with the other components to allow complex applications to be built of simple building blocks. ReactJS uses virtual DOM based mechanism to fill data in HTML DOM. The virtual DOM works fast as it only changes individual DOM elements instead of reloading complete DOM every time.

Components are the building blocks of any React app. Components allow us to split the UI into independent and reusable pieces. A component is a combination of:

1. Template using HTML
2. User Interactivity using JS
3. Applying Styles using CSS

Backend: Python

Backend Connectivity: Flask

Flask is a lightweight and versatile web application framework for Python. Below are the key features of the Flask:

1. **Minimalistic:** Flask follows a "micro" framework philosophy, which means it provides only the essentials for building web applications. This allows developers to have more flexibility and control over their codebase.
2. **Routing:** Flask uses a routing system to map URLs to functions. Developers can define routes using decorators like `@app.route('/path')` to specify which function should handle requests to specific URLs.
3. **Extensions:** Flask has a rich ecosystem of extensions that add extra functionality to the framework. These extensions cover a wide range of features, including database integration, authentication, form handling, and more.
4. **WSGI Compliance:** Flask is WSGI (Web Server Gateway Interface) compliant, which means it can run on any WSGI-compatible web server. This flexibility allows developers to deploy Flask applications using various hosting options.
5. **Community and Documentation:** Flask has a large and active community of developers who contribute plugins, tutorials, and support through forums and documentation. The official Flask documentation is comprehensive and well-maintained, making it easy for developers to get started and find answers to their questions.

Database: MySQL

MySQL is an open-source relational database management system (RDBMS) that is widely used for managing and storing structured data. Some of its key features are:

- Relational Database
- Open Source
- Cross-Platform
- Scalability
- High Performance
- Replication and High availability
- Security
- Community and Support

APIs used: Google login, (speech to voice)

Frontend:

Main Page

```
import React, { useState } from 'react';
import { BrowserRouter as Router, Routes, Route, Link, useLocation,
useNavigate } from 'react-router-dom';
import './App.css'; // Import your CSS file for styling
import logo from './Delphi.png';
import Login from './Login';
import Signup from './Signup';
import Voice from './Voice';
import Forgot from './frgtpass.js';

// Header Component
export const Header = ({ isAuthenticated, setIsAuthenticated }) => {
  return (
    <header className="header">
      <div className="logo">
        {/* Your logo */}
      </div>
      <nav className="navigation">
        <ul>
          {isAuthenticated ? (
            <li><Link to="/">Logout</Link></li>
          ) : (
```

```

        <>
        <li><Link to="/login">Login</Link></li>
        <li><Link to="/signup">Sign Up</Link></li>
        </>
      )}
    </ul>
  </nav>
</header>
);
};
// Footer Component
export const Footer = () => {
  return (
    <footer className="footer">
      <ul>
        <li>
          <button onClick={() => {
            alert("Created By: Deekshita Athreya and Vashita Darji");
          }}>
            Know More
          </button>
        </li>
        <li><a href="mailto:22bce062@nirmauni.ac.in">Contact us</a></li>
      </ul>
    </footer>
  );
};

// Main Content Component
const MainContent = ({ isAuthenticated, setIsAuthenticated }) => {
  return (
    <div className='main-content'>
      <div className="logo">
        { /* Your logo */ }
      </div>
      <nav className="navigation">
        <ul>
          {isAuthenticated ? (
            <li><Link to="/">Logout</Link></li>
          ) : (
            <>
            <li><Link to="/login">Login</Link></li>
            <li><Link to="/signup">Sign Up</Link></li>
            </>
          )}
        </ul>
      </nav>
    </div>
  );
};

```

```

    })
    </ul>
  </nav>
  <div className='text'>
    <h3>Step into the Realm of Delphi</h3>
    <h5>Where Ancient Wisdom Meets Modern Assistance.</h5>
    <Link to="/login" className="get-started-button btn-glow">Let's
Get Started</Link>
  </div>
</div>
);
};

// App Component
const App = () => {

  return (
    <Router>
      <div className="app">
        <Routes>
          <Route exact path="/" element={
            <div>
              <MainContent />
              <Footer />
            </div>
          } />
          <Route path="/login" element={<Login />} />
          <Route path="/signup" element={<Signup />} />
          <Route path="/voice" element={<Voice />} />
          <Route path="/forgot" element={<Forgot />} />
        </Routes>
      </div>
    </Router>
  );
};

export default App;

```

Imports: The first section imports necessary modules and components. React is imported along with useState for managing component state. BrowserRouter, Routes, Route, Link, useLocation, and useNavigate are components and hooks

from react-router-dom used for routing. Other components like Login, Signup, Voice, and Forgot are imported from their respective files.

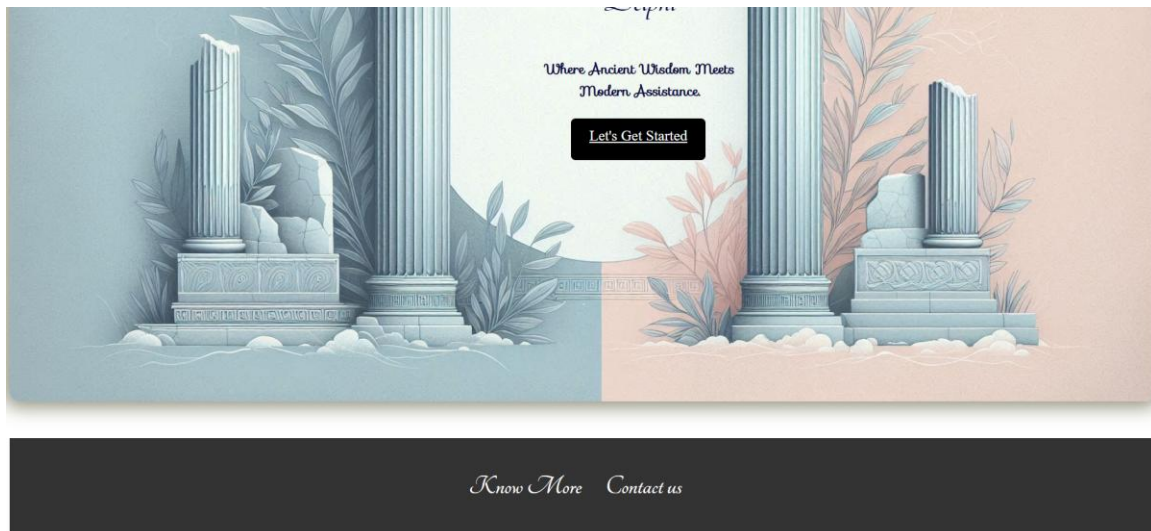
Header Component: This is a functional component Header which takes `isAuthenticated` and `setIsAuthenticated` as props. It renders a header section with a logo and navigation links. If the user is authenticated, it shows a logout link; otherwise, it shows login and signup links.

Footer Component: The Footer component is another functional component rendering a footer section with a button to know more about the creators and a link to contact them via email.

MainContent Component: This component renders the main content of the page. It contains a logo, navigation links, and a welcome message with a button to get started, which redirects to the login page.

App Component: This is the main App component. It wraps the entire application with Router and defines different routes using Route components. Depending on the route, it renders either the MainContent, Login, Signup, Voice, or Forgot component.





Login Page:

```
import React, { useState } from 'react';
import './Login.css'; // Import your CSS file for styling
import { GoogleLogin } from '@react-oauth/google';
import { useNavigate } from 'react-router-dom';

const AncientGreekLogin = () => {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [failed, setFailed] = useState('');
  const navigate = useNavigate();

  const handleSubmit = async (e) => {
    e.preventDefault(); // Prevent form submission

    try {
      const response = await fetch('/login', {
        method: 'POST',
        headers: {
          'Content-Type': 'application/json'
        },
        body: JSON.stringify({ e: email, p: password }) // Simplified
        object shorthand
      });

      if (!response.ok) {
        throw new Error('Network response was not ok');
      }
    }
  }
}
```

```

    const data = await response.json();
    if (data.response === 1) {
      navigate('/voice');
    } else {
      setFailed('Please enter valid credentials');
    }
  } catch (error) {
    console.error('Error:', error);
  }
};

const responseMessage = (response) => {
  console.log(response);
  navigate('/voice');
};

const errorMessage = (error) => {
  console.log(error);
};

return (
  <div className="box-form">
    <div className="left">
      <div className="overlay">
        <p className='tag'>"In Delphi, ancient wisdom whispers through
the ages, guiding us on a timeless journey of enlightenment."</p>
        <span className='gl'>
          <center>
            <GoogleLogin onSuccess={responseMessage}
onError={errorMessage} />
          </center>
        </span>
      </div>
    </div>
    <div className="right">
      <h5>Login</h5>
      <p style={{ color: 'red', fontWeight: 'bold' }}><failed></p>
      <a className='home home-glow' href="/">Home</a>
      <p>Don't have an account? <a href="/signup">Create One</a>, it
takes less than a minute!</p>
      <form onSubmit={handleSubmit}>

```

```

        <div className="inputs">
          <input type="text" placeholder="Email" value={email}
onChange={(e) => setEmail(e.target.value)} />
          <br />
          <input type="password" placeholder="Password" value={password}
onChange={(e) => setPassword(e.target.value)} />
        </div>

        <br />

        <div className="remember-me--forget-password">
          <span>
            <a href="/forgot"><i className="fa fa-twitter" aria-
hidden="true"></i>Forgot Password?</a>
          </span>
        </div>
        <br />
        <button className='submit' type="submit">Login</button>
      </form>
    </div>
  </div>
);
};

export default AncientGreekLogin;

```

1. Imports:

- React and useState are imported from 'react'.
- The component's CSS file, 'Login.css', is imported for styling.
- GoogleLogin component is imported from '@react-oauth/google' for Google OAuth authentication.
- useNavigate hook is imported from 'react-router-dom' for programmatic navigation.

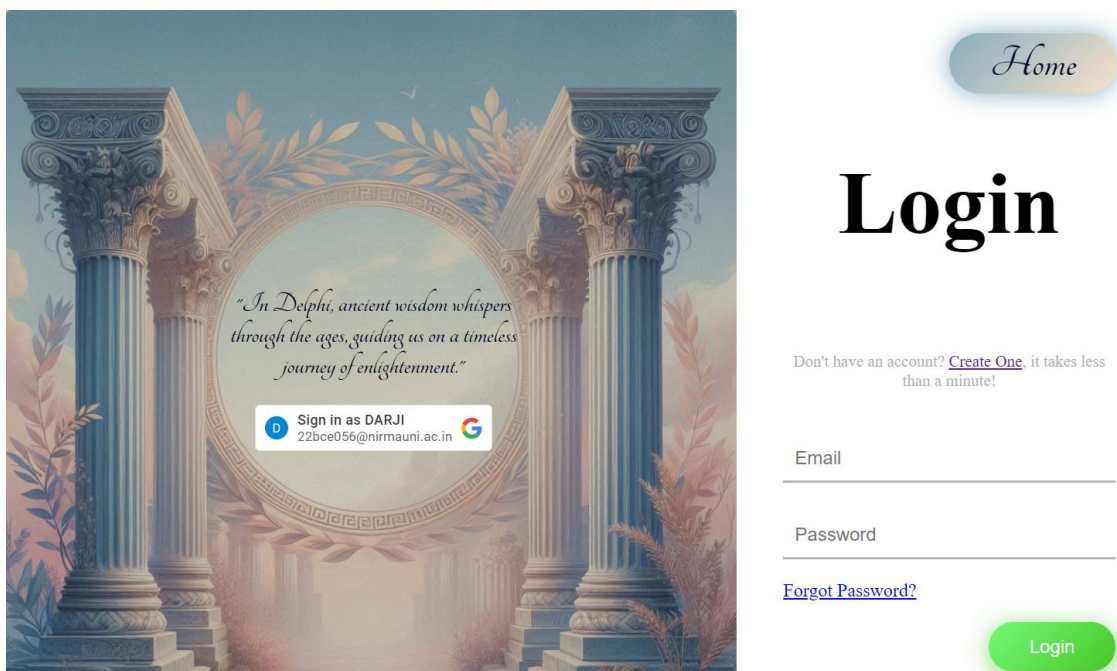
2. Component Functionality:

- State variables are initialized using the useState hook:
 - email and password for storing user input.
 - failed for displaying login failure messages.
- The navigate function from useNavigate hook is used for programmatic navigation.

3. handleSubmit Function:

- This function is called when the login form is submitted.

- It prevents the default form submission behavior using `e.preventDefault()`.
 - It sends a POST request to `/login` endpoint with user credentials (email and password).
 - If the response indicates successful login (`data.response === 1`), it navigates the user to the `/voice` page.
 - Otherwise, it sets the failed state to display a login failure message.
- 4. `responseMessage` and `errorMessage` Functions:**
- These functions handle the success and error callbacks of the Google login component respectively.
 - If the Google login is successful, `responseMessage` navigates the user to the `/voice` page.
 - Any errors during Google login are logged to the console by `errorMessage`.
- 5. `Render Method`:**
- The render method returns JSX representing the login page layout.
 - It consists of two main sections:
 - The left section displays a quote and a Google login button.
 - The right section contains the login form, including input fields for email and password, a login button, and links for home, signup, and forgot password pages.



Signin Page:

```
import React, { useState } from 'react';
import './Signup.css'; // Import your CSS file for styling
import { GoogleLogin } from '@react-oauth/google';
import { useNavigate } from 'react-router-dom';

const AncientGreekLogin = () => {
  const [name, setName] = useState('');
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [failed, setFailed] = useState('');
  const navigate = useNavigate();

  const handleSubmit = async (e) => {
    e.preventDefault(); // Prevent form submission

    // Check if any field is empty
    if (!name || !email || !password) {
      setFailed('Please fill in all fields');
      return;
    }

    try {
      const response = await fetch('/signup', {
        method: 'POST',
        headers: {
          'Content-Type': 'application/json'
        },
        body: JSON.stringify({ 'n': name, 'e': email, 'p': password }) //
        Simplified object shorthand
      });

      if (!response.ok) {
        throw new Error('Network response was not ok');
      }

      const data = await response.json();
      if (data.response === 'registered') {
        alert('Successfully Registered! Redirecting to the login page');
        navigate('/login');
      } else if (data.response === 'exists'){
```

```

        setFailed('The username is already registered!');
    }
    else{
        setFailed('An unexpected error occurred!')
    }
} catch (error) {
    console.error('Error:', error);
}
};

const responseMessage = (response) => {
    console.log(response);
    navigate('/voice');
};

const errorMessage = (error) => {
    console.log(error);
};

return (
    <div className="box-forms">
        <div className="rights">
            <h5>Register</h5>
            <p style={{ color: 'red', fontWeight: 'bold' }}>{failed}</p>
            <a className='homes homes-glow' href="/">Home</a>
            <p>Already have an account? <a href="/login">Back to login</a></p>
            <h3> Join the echelons of wisdom today!</h3>
            <form onSubmit={handleSubmit}>
                <div className="inputss">
                    <input type="text" placeholder="Name" value={name}
onChange={(e) => setName(e.target.value)} />
                    <input type="text" placeholder="Email" value={email}
onChange={(e) => setEmail(e.target.value)} />
                    <br />
                    <input type="password" placeholder="Password" value={password}
onChange={(e) => setPassword(e.target.value)} />
                </div>

                <br />
                <br />
                <button className='submits' type="submit">Sign Up</button>
            </form>
        </div>
    </div>

```

```

    <div className="lefts">
      <div className="overlays">
        <p className='tags'>"Delphi, guardian of timeless whispers,
guiding your journey through the echoes of antiquity."</p>
        <span className='gls'>
          <center>
            <GoogleLogin onSuccess={responseMessage}
onError={errorMessage} style={{ width:'200px'}} />
          </center>
        </span>
      </div>
    </div>
  </div>
);
};

export default AncientGreekLogin;

```

1. Imports:

- React and useState are imported from 'react'.
- The component's CSS file, 'Signup.css', is imported for styling.
- GoogleLogin component is imported from '@react-oauth/google' for Google OAuth authentication.
- useNavigate hook is imported from 'react-router-dom' for programmatic navigation.

2. Component Functionality:

- State variables are initialized using the useState hook:
 - name, email, and password for storing user input.
 - failed for displaying signup failure messages.
- The navigate function from useNavigate hook is used for programmatic navigation.

3. handleSubmit Function:

- This function is called when the signup form is submitted.
- It prevents the default form submission behavior using e.preventDefault().
- It checks if any field is empty and sets the failed state if so.
- It sends a POST request to '/signup' endpoint with user details (name, email, and password).
- Depending on the response, it either alerts the user of successful registration and navigates them to the login page, or sets the failed state with appropriate error messages.

4. responseMessage and errorMessage Functions:

- These functions handle the success and error callbacks of the Google login component respectively.
- If the Google login is successful, `responseMessage` navigates the user to the `'/voice'` page.
- Any errors during Google login are logged to the console by `errorMessage`.

5. Render Method:

- The render method returns JSX representing the signup page layout.
- It consists of two main sections:
 - The right section contains the signup form, including input fields for name, email, and password, a signup button, and links for home and login pages.
 - The left section displays a quote and a Google login button.

[Home](#)

Register

Already have an account? [Back to login](#)

Join the echelons of wisdom today!

Name

Email

Password

*"Delphi, guardian of timeless whispers,
guiding your journey through the echoes of
antiquity."*

Sign in as DARJI

22bce056@nirmauni.ac.in

Forgot Password Page:

```

import React, { useState } from 'react';
import './frgtpass.css'; // Import your CSS file for styling
import { useNavigate } from 'react-router-dom';

const ForgotPassword = () => {
  const [name, setName] = useState('');
  const [email, setEmail] = useState('');
  const [newPassword, setNewPassword] = useState('');
  const [resetOptionVisible, setResetOptionVisible] = useState(false);
  const [failed, setFailed] = useState('');
  const [submitButtonText, setSubmitButtonText] = useState('Submit');
  const navigate = useNavigate();

  const handleSubmit = async (e) => {
    e.preventDefault(); // Prevent form submission

    // Check if any of the fields are empty
    if (!name || !email) {
      setFailed('Please enter your name and email address');
      return;
    }
    try {
      const response = await fetch('/forgot', {
        method: 'POST',
        headers: {
          'Content-Type': 'application/json'
        },
        body: JSON.stringify({ 'n': name, 'e': email }) // Simplified object shorthand
      });

      if (!response.ok) {
        throw new Error('Network response was not ok');
      }

      const data = await response.json();
      if (data.response === 'exists'){
        setResetOptionVisible(true);
        setSubmitButtonText('Reset Password');
      }
      else{
        setFailed('An unexpected error occurred!')
      }
    } catch (error) {

```

```

        console.error('Error:', error);
    }
};

const handleResetPassword = async () => {
    try {
        const response = await fetch('/reset', {
            method: 'POST',
            headers: {
                'Content-Type': 'application/json'
            },
            body: JSON.stringify({ 'p': newPassword, 'e': email }) // Include
email in the body for reset endpoint
        });

        if (!response.ok) {
            throw new Error('Network response was not ok');
        }

        const data = await response.json();
        if (data.response === 1) {
            alert('Password reset successfully! Redirecting to login page');
            navigate('/login');
        }
        else {
            setFailed('An unexpected error occurred!')
        }
    } catch (error) {
        console.error('Error:', error);
    }
};

return (
    <div className="box-formf">
        <div className="rightf">
            <h5 style={{marginBottom: '0px'}}>Forgot</h5>
            <h5>Password</h5>
            <p style={{ color: 'red', fontWeight: 'bold' }}>{failed}</p>
            <a className='homef' href="/">Home</a>
            <p>Remembered your password? <a href="/login">Back to
login</a></p>
            <form onSubmit={handleSubmit}>
                <div className="inputsf" id='inputs'>

```

```

        <input type="text" placeholder="Enter your name" value={name}
onChange={(e) => setName(e.target.value)} />
        <input type="text" placeholder="Enter your email"
value={email} onChange={(e) => setEmail(e.target.value)} />
        {resetOptionVisible && (
            <input type="password" placeholder="Enter your password"
value={newPassword} onChange={(e) => setNewPassword(e.target.value)} />
        )}
    </div>

    <br />
    <br />
    {resetOptionVisible ? (
        <button type="button" className='submitf'
onClick={handleResetPassword}>{submitButtonText}</button>
    ) : (
        <button className='submitf' type="submit">Submit</button>
    )}
    </form>
</div>

<div className="leftf">
    <div className="overlayf">
        <span>
            {/* You can add any additional content here if needed */}
        </span>
    </div>
</div>
</div>
);
};

export default ForgotPassword;

```

1. Imports:

- React and useState are imported from 'react'.
- The component's CSS file, 'frgtpass.css', is imported for styling.
- useNavigate hook is imported from 'react-router-dom' for programmatic navigation.

2. Component Functionality:

- State variables are initialized using the useState hook:
 - name, email, and newPassword for storing user input.
 - resetOptionVisible to control the visibility of the password reset option.

- failed for displaying failure messages.
 - submitButtonText to control the text of the submit button.
- The navigate function from useNavigate hook is used for programmatic navigation.

3. handleSubmit Function:

- This function is called when the form is submitted.
- It prevents the default form submission behavior using e.preventDefault().
- It checks if name and email fields are empty and sets the failed state if so.
- It sends a POST request to '/forgot' endpoint with user name and email.
- Depending on the response, it either sets resetOptionVisible to true and changes the submit button text to 'Reset Password', or sets the failed state with an appropriate error message.

4. handleResetPassword Function:

- This function is called when the reset password button is clicked.
- It sends a POST request to '/reset' endpoint with the new password and email.
- Depending on the response, it either alerts the user of successful password reset and navigates them to the login page, or sets the failed state with an appropriate error message.

5. Render Method:

- The render method returns JSX representing the forgot password page layout.
- It consists of two main sections:
 - The right section contains the forgot password form, including input fields for name and email, and optionally for a new password if resetOptionVisible is true. It also includes a submit button, which may change to a reset password button based on resetOptionVisible.
 - The left section is currently empty, but you can add any additional content if needed.

Home

Forgot Password

Remembered your password? [Back to login](#)

Enter your name

Enter your email



Voice Page:

```
import React, { useState, useEffect, useRef } from 'react';
import './Voice.css'; // Import your CSS file for styling
import mic from './mic.png';
import initialVideoFile from './video.mp4'; // Import your initial video
file
import {Header, Footer} from "../App";

const Voice = () => {
  const isAuthenticated = true;
  const [transcript, setTranscript] = useState('');
  const [serverResponse, setServerResponse] = useState('');
  const [typedText, setTypedText] = useState('');
  const [videoUrl, setVideoUrl] = useState(initialVideoFile); // Set
initial video URL
  const [listening, setListening] = useState(false); // State to track
if recognition is active
  const videoRef = useRef(null);
  const ansRef = useRef(null);
  const recognitionRef = useRef(null); // Ref to store recognition
instance
```

```

    useEffect(() => {
        recognitionRef.current = new (window.SpeechRecognition ||
window.webkitSpeechRecognition || window.mozSpeechRecognition ||
window.msSpeechRecognition)();
        recognitionRef.current.lang = 'en-US';

        // Cleanup function to stop recognition when component unmounts or
before starting a new one
        return () => {
            if (recognitionRef.current) {
                recognitionRef.current.stop();
            }
        };
    }, []);

    useEffect(() => {
        // Play initial video when the component mounts
        if (videoRef.current) {
            // Check if the video is paused to avoid playing it multiple
times
            if (videoRef.current.paused) {
                videoRef.current.play().catch(error => {
                    console.error('Error playing video:', error);
                });
            }
        }
    }, []);

    const startRecognition = () => {
        setTranscript(''); // Clear the transcript
        setTypedText(''); // Clear the typed text
        setServerResponse(''); // Clear the server response

        if (recognitionRef.current) {
            // Toggle listening state
            setListening(true);

            // Stop previous recognition instance before starting a new
one
            recognitionRef.current.stop();
            recognitionRef.current.onresult = function(event) {
                const transcript = event.results[0][0].transcript;
                var transcript1 = transcript.charAt(0).toUpperCase() +
transcript.slice(1);

```

```

        setTranscript(transcript1);
        fetch('/process', {
            method: 'POST',
            headers: {
                'Content-Type': 'application/json'
            },
            body: JSON.stringify({ transcript: transcript })
        })
        .then(response => response.json())
        .then(data => {
            setServerResponse(data.response);
            if (data.video) {
                console.log('Received video URL:', data.video); //
Log received video URL
                // Set video source dynamically by updating the
source element directly
                const sourceElement =
document.getElementById('src');
                sourceElement.src = data.video;
                videoRef.current.load();
                videoRef.current.play(); // Reload the video
element
            } else {
                console.log('No video URL received.');
```

```

function simulateTyping(text) {
  let index = 0;
  const typingInterval = setInterval(() => {
    setTypedText(prevText => prevText + text[index]);
    index++;
    if (index === text.length) {
      clearInterval(typingInterval);
    }
  }, 50); // Adjust typing speed here
}

return (
  <div>
    <div className="box-form-v">
      <a className='homes' href='/'>Home</a>
      <link
href="https://fonts.googleapis.com/css2?family=Rubik+Scribble&family=Teko:wght@300..700&display=swap" rel="stylesheet"></link>
      <link
href="https://fonts.googleapis.com/css2?family=GFS+Didot&display=swap" rel="stylesheet"></link>
      <div className='mic'>
        <img src={mic} alt="Mic" className='img' />
      </div>
      {/* Conditionally render button text based on listening state */}
      <button className="btn btn-gradient btn-glow"
onClick={startRecognition}>
        {listening ? "Listening..." : "Start Speaking!"}
      </button>
      <div className='circle'>
        <div className="circular-video" id='video-container'>
          <video id='video' ref={videoRef} onEnded={() =>
setServerResponse('')} preload="metadata">
            <source src={videoUrl} type="video/mp4"
id='src'></source>
          </video>
        </div>
      </div>
      <div className='ques' id='ques'>{transcript}</div>
      <div className='ans' id='ans'
ref={ansRef}>{typedText}</div>
    </div>
    <Footer />
  </div>
)

```

```

    </div>
  );
};

export default Voice;

```

1. Imports:

- React, useState, useEffect, and useRef are imported from 'react'.
- The component's CSS file, 'Voice.css', is imported for styling.
- An image file for the microphone icon (mic.png) and an initial video file (initialVideoFile) are imported.

2. Component Functionality:

- State variables are initialized using the useState hook:
 - transcript for storing the recognized speech transcript.
 - serverResponse for storing the server response to the recognized speech.
 - typedText for simulating typing effect based on the server response.
 - videoUrl for storing the URL of the video to be displayed.
 - listening to track if the recognition is active.
- The useRef hook is used to create references to the video element (videoRef) and the answer element (ansRef).
- The useEffect hook is used to set up the SpeechRecognition API and play the initial video when the component mounts.

3. startRecognition Function:

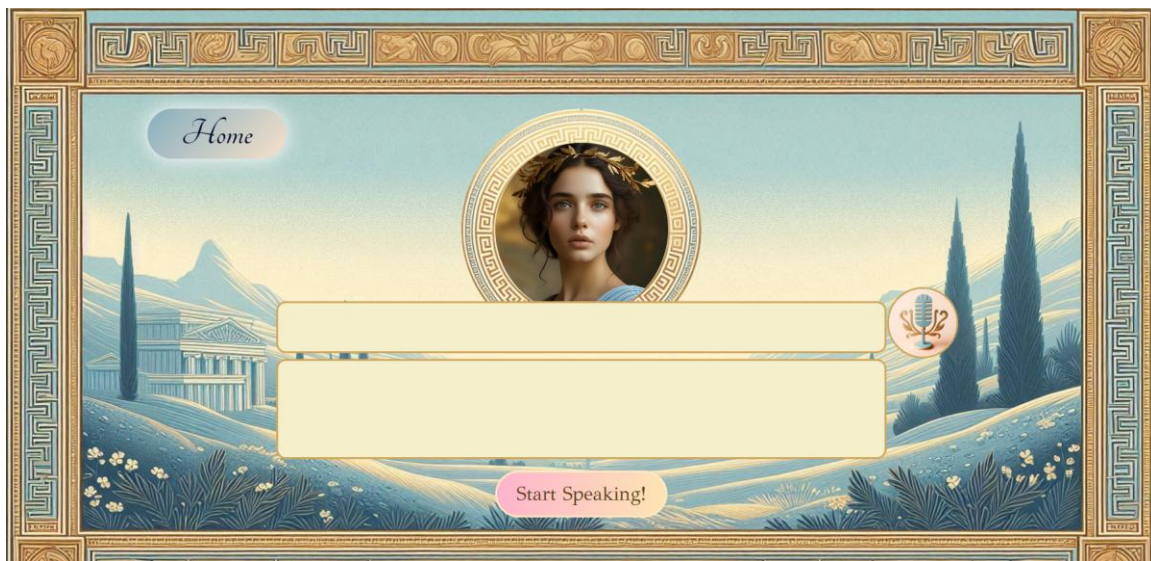
- This function is called when the user clicks the button to start recognition.
- It initializes the SpeechRecognition API, sets event handlers for recognition events (e.g., onresult, onerror, onend), and starts recognition.
- When recognition results are received, it updates the transcript state, sends the transcript to the server for processing, and updates the serverResponse state.
- If a video URL is received from the server response, it dynamically updates the video source and plays the video.
- It also calls the simulateTyping function to simulate typing the server response.

4. simulateTyping Function:

- This function simulates typing the server response character by character.
- It uses a setInterval to gradually append characters of the server response to the typedText state.

5. Render Method:

- The render method returns JSX representing the voice recognition page layout.
- It consists of a microphone icon, a button to start recognition, a video player, and areas to display the recognized speech transcript (transcript) and the simulated typing effect (typedText).
- The button text changes based on the listening state.
- The video source is dynamically updated based on the videoUrl state.
- The footer is rendered below the voice recognition feature.



Backend:

```
from flask import Flask, request, jsonify
import mysql.connector
import json
import subprocess
import webbrowser
import speech_recognition as sr
import wikipedia
import datetime
import os
import wolframalpha
import random
```

```

from gtts import gTTS
import os
import sync
import re

app = Flask(__name__)
def b(text):
    return re.sub(r'\(.*?\)', '', text)
@app.route('/process', methods=['POST'])
def process():
    data = request.json
    transcript = data['transcript']
    try:
        if 'time' in transcript:
            now=datetime.datetime.now()
            x=now.strftime('%H:%M:%S')
            timestr='The time right now is'+x
            ans=timestr

        elif 'date' in transcript:
            now=datetime.datetime.now()
            x=now.strftime('%d/%m/%Y')
            timestr='Today\'s date is'+x
            ans=timestr

        #gives weather of place in query. If no place mentioned,
        gives weather based on device location.

        elif 'weather' in transcript or 'climate' in transcript:
            api='VEL7AU-KEQT69L5QK'
            client=wolframalpha.Client(api)
            res=client.query(transcript)
            weather=next(res.results).text
            ans=weather

        #to search for personalities on wikipedia

        elif 'who is' in transcript or 'who was' in transcript :
            li=transcript.split()
            searchfor=str(li[2:])
            result=wikipedia.summary(searchfor,
sentences=2)

            ans=b(result)

```

```

        #to search for places on wikipedia

        elif 'where is' in transcript or 'where was' in transcript
:
            li=transcript.split()
            searchfor=str(li[2:])
            result=wikipedia.summary(searchfor,
sentences=2)

            ans=b(result)

        #to search for events or objects on wikipedia and also do
calculations

        elif 'what is' in transcript or 'what was' in transcript :
            operators=['plus', 'minus','into','multiplied
by','divided by','upon','raise to','square root','cube root','+','-
','*','/','**','^','calculate']

            #to figure out if user wants calculations or
wikipedia search. If any elements of operators list present in user
transcript, then end the checking and proceed to calculations.

            for element in range(0,len(operators)):
                if str(operators[element]) in transcript:
                    do='calculations'
                    break
                else:
                    do='wikipedia'

            if do=='wikipedia':
                li=transcript.split()
                searchfor=str(li[2:])
                result=wikipedia.summary(searchfor,
sentences=2)

                ans=b(result)

            #various operations

            elif do=='calculations':
                li=transcript.split()
                if 'plus' in transcript or '+' in
transcript:

                    no1=int(li[2])
                    no2=int(li[4])

```

```

        ans=no1+no2
    elif 'minus' in transcript or '-' in
transcript:
        no1=int(li[2])
        no2=int(li[4])
        ans=no1-no2
    elif 'into' in transcript or '*' in
transcript:
        no1=int(li[2])
        no2=int(li[4])
        ans=no1*no2
    elif 'multiplied by' in transcript:
        no1=int(li[2])
        no2=int(li[5])
        ans=no1*no2
    elif '/' in transcript:
        no1=int(li[2])
        no2=int(li[4])
        ans=no1/no2
    elif 'divided by' in transcript:
        no1=int(li[2])
        no2=int(li[5])
        ans=no1/no2
    elif 'raise to' in transcript:
        no1=int(li[2])
        no2=int(li[5])
        ans=no1**no2
    elif '**' in transcript or '^' in
transcript:
        no1=int(li[2])
        no2=int(li[4])
        ans=no1**no2
    elif 'square root of ' in
transcript:
        no1=int(li[-1])
        ans=no1**(0.5)
    elif 'cube root' in transcript:
        no1=int(li[-1])
        ans=no1**(1/3)
    answerstr='Your answer is : ' +
str(ans)
    ans=answerstr

```

#to search on web

```

        elif 'search for' in transcript:
            li=transcript.split()
            search=' '.join(li[2:])
            announce2='Search results for your query are:'
            ans=announce2
            webbrowser.open('https://www.google.com/search
?q='+search, new=1)
            #to play videos from youtube

        elif 'play' in transcript:
            li=transcript.split()
            search=' '.join(li[1:])
            announce2='Search results for your query are:'
            ans=announce2
            webbrowser.open('www.youtube.com/results?search_query=' + search,new=1)

            #reply to various forms of greeting

        elif 'hello' in transcript or 'hi' in transcript:
            list=['Hello','Hi! how are you?', 'hey!! How
are you?','hii! How are you doing?']
            greetingstr=random.choice(list)
            ans=greetingstr

        elif 'how are you' in transcript or 'how do you do' in
transcript or 'what are you doing' in transcript:
            list=['Never been better', 'Having fun!Its
always a pleasure to be with you']
            greeting1str=random.choice(list)
            ans=greeting1str

            #Message to be displayed in case of an error

    except:
        errorstr='Sorry, something went wrong. Please try again'
        ans=errorstr
        tts = gTTS(text=ans, lang="en-IN")

        tts.save('audio.mp3')
        link=sync.animate()
        # Convert the response to a string
        processed_data = {'response': ans,'video':link}
        response_string = json.dumps(processed_data)

```

```

        return response_string

@app.route('/login', methods=['GET','POST'])
def login():
    data = request.get_json() # Get data from the request body
    email = data['e']
    password = data['p']
    connection = mysql.connector.connect(
        host="localhost",
        user="root",
        password="Athreya2608#",
        database="fswd_project"
    )
    cursor = connection.cursor()
    sql_query = "SELECT email, password FROM users WHERE email = %s AND
password = %s"

    # Execute the query with values
    cursor.execute(sql_query, (email, password))

    # Fetch one row (if exists)
    row = cursor.fetchone()

    if row:
        return jsonify({'response': 1, 'email': email})
    else:
        return jsonify({'response': 0})

@app.route('/signup', methods=['GET','POST'])
def signup():
    data = request.get_json() # Get data from the request body
    name = data['n']
    email = data['e']
    password = data['p']

    # Connect to MySQL database
    connection = mysql.connector.connect(
        host="localhost",
        user="root",
        password="Athreya2608#",
        database="fswd_project"
    )
    cursor = connection.cursor()

```

```

    # Check if email already exists
    cursor.execute("SELECT * FROM users WHERE email = %s", (email,))
    existing_user = cursor.fetchone()

    if existing_user:
        # If email exists, return a response indicating that the user is
        already registered
        return jsonify({'response': 'exists', 'email': email}), 200

    # Insert new user if email is not found
    sql_query = "INSERT INTO users (name, email, password) VALUES (%s, %s,
%s)"
    try:
        cursor.execute(sql_query, (name, email, password))
        connection.commit()
        return jsonify({'response': 'registered', 'email': email}), 200
    except mysql.connector.Error as err:
        # Handle MySQL errors
        return jsonify({'response': 'error', 'error': str(err)}), 500
    finally:
        cursor.close()
        connection.close()

@app.route('/forgot', methods=['GET', 'POST'])
def forgot():
    data = request.get_json() # Get data from the request body
    name = data['n'].lower()
    email = data['e']

    # Connect to MySQL database
    connection = mysql.connector.connect(
        host="localhost",
        user="root",
        password="Athreya2608#",
        database="fswd_project"
    )
    cursor = connection.cursor()

    # Check if email already exists
    cursor.execute("SELECT * FROM users WHERE email = %s and name=%s",
(email,name))
    existing_user = cursor.fetchone()

    if existing_user:

```

```

        # If email exists, return a response indicating that the user is
already registered
        return jsonify({'response': 'exists', 'email': email}), 200
    else:
        return jsonify({'response': 'notexists', 'email': email}), 200
@app.route('/reset', methods=['GET','POST'])
def reset():
    data = request.get_json() # Get data from the request body
    password=data['p']
    email=data['e']

    # Connect to MySQL database
    connection = mysql.connector.connect(
        host="localhost",
        user="root",
        password="Athreya2608#",
        database="fswd_project"
    )
    cursor = connection.cursor()

    sql_query = "UPDATE users set password=%s where email=%s"
    try:
        cursor.execute(sql_query, (password, email))
        connection.commit()
        return jsonify({'response': 1, 'email': email}), 200
    except mysql.connector.Error as err:
        # Handle MySQL errors
        return jsonify({'response': 0, 'error': str(err)}), 500
    finally:
        cursor.close()
        connection.close()
if __name__ == '__main__':
    app.run(debug=True)

```

1. Imports:

- The required libraries and modules are imported, including Flask for creating the web application, modules for database interaction (mysql.connector), speech recognition (speech_recognition), querying WolframAlpha (wolframalpha), interacting with Wikipedia (wikipedia), and others for miscellaneous tasks.

2. Flask App Setup:

- An instance of the Flask application is created.

- The /process, /login, /signup, /forgot, and /reset routes are defined for different functionalities.
- 3. process Route:**
 - This route handles processing of voice commands received from the frontend.
 - The transcript of the voice command is extracted from the request JSON.
 - The transcript is analyzed to perform various actions such as retrieving the current time, weather information, searching Wikipedia, performing calculations, searching the web, playing YouTube videos, and responding to greetings.
 - The response is converted to speech using Google Text-to-Speech (gTTS) and saved as an audio file.
 - The response is then sent back to the frontend along with a link to the generated video file.
 - 4. login Route:**
 - This route handles user login functionality.
 - User credentials (email and password) are retrieved from the request JSON.
 - A MySQL database is queried to check if the user exists with the provided credentials.
 - A response indicating success or failure is returned to the frontend.
 - 5. signup Route:**
 - This route handles user registration functionality.
 - User details (name, email, and password) are retrieved from the request JSON.
 - The user's email is checked to ensure it does not already exist in the database.
 - If the email is unique, the user is registered, and a success response is returned. Otherwise, an error response is returned.
 - 6. forgot Route:**
 - This route handles the forgot password functionality.
 - User details (name and email) are retrieved from the request JSON.
 - The database is queried to check if the user exists with the provided name and email.
 - A response indicating whether the user exists or not is returned.
 - 7. reset Route:**
 - This route handles resetting the user's password.
 - New password and email are retrieved from the request JSON.
 - The user's password is updated in the database.
 - A success or error response is returned based on the outcome.
 - 8. Main Block:**
 - The Flask application is run with debugging enabled.

Conclusion:

In our project the use of React JS along with HTML and CSS in the frontend, MySQL for the database, Flask for backend connectivity and python for the backend has provided us with the overview of how to integrate and use different technologies for making an efficient project. Not only it provided us with the various new functions and aspects of using these technologies. React JS helped us to understand the different components as well as the concepts of Real and Virtual DOM. React also helped us in understanding the concepts of mounting and rendering the components. Moreover, through Flask, which a lightweight python-based connector made it easy for us to host our webpage. It also gave us experience of using different APIs and provided hands on experience.

Future Expansion:

Our project can be further expanded using following:

- Training AI model for key words extraction
- Can expand this with using AI which can identify itself which page to search to i.e. Wikipedia, YouTube, etc.
- It can also be expanded using NLP(Natural Language Processing)
- It can be expanded in other languages instead of only English
- We can also create our own database which will reduce the search time and improve efficiency and scalability.

References:

1. <https://webdamn.com/login-and-registration-with-python-flask-mysql/>
2. <https://www.google.com/search?q=>
3. www.youtube.com/results?search_query=
4. <https://www.wolframalpha.com/>
5. <https://www.geeksforgeeks.org/how-to-connect-reactjs-with-flask-api/>
6. https://www.w3schools.com/python/python_mysql_getstarted.asp
7. <https://gooey.ai/Lipsync/>
8. [Image Creator from Microsoft Designer \(bing.com\)](#)
9. <https://developers.google.com/identity/protocols/oauth2>
10. <https://stackoverflow.com/questions/73499049/how-to-use-header-footer-in-reactjs>

